

Application of `abmneo()` function to digestate storage

Sasha D. Hafner

19 August, 2026 11:42

Parameters

```
grp_VS_dig3 <- list(  
  grps = c('m1', 'm2', 'm3', 'm4', 'm5'),  
  yield = c(default = 0.05),  
  xa_fresh = c(default = 0.06),  
  xa_init = c(default = 0.06),  
  d_max = c(default = 0.02),  
  qhat_opt = c(m1 = 1.6, m2 = 2.8, m3 = 6.2, m4 = 10, m5 = 16),  
  T_opt = c(m1 = 18, m2 = 28, m3 = 36, m4 = 44, m5 = 55),  
  T_min = c(m1 = 2, m2 = 2, m3 = 15, m4 = 26, m5 = 30),  
  T_max = c(m1 = 25, m2 = 38, m3 = 45, m4 = 51, m5 = 60),  
  ksmat = matrix(  
    c(1.1, 1.1, 1.1, 1.1, 1.1),  
    nrow = 1,  
    byrow = TRUE,  
    dimnames = list(  
      c('VFA'),  
      c('m1', 'm2', 'm3', 'm4', 'm5')  
    )  
  ),  
  mstoich = matrix(  
    c(  
      -1, -1, -1, -1, -1,  
      1, 1, 1, 1, 1  
    ),  
    nrow = 2,  
    byrow = TRUE,  
    dimnames = list(  
      c('VFA', 'CH4'),  
      c('m1', 'm2', 'm3', 'm4', 'm5')  
    )  
  )  
)  
  
sub_VS_dig3 <- list(  
  subs = 'PS',  
  sub_enrich = 'PS',  
  xd = 'xd',  
  h_rate_ref = c(default = 0.1),  
  h_rate_q10 = c(default = 2),
```

```

    sub_fresh = c(PS = 18.22, xd = 0),
    sub_init  = c(PS = 18.22, xd = 0)
)

inf_VS_dig3 <- list(
  VFA_fresh = 0.5,
  VFA_init  = 0.5,
  pH        = 8,
  dens      = 1000
)

```

With time series inputs.

```

mass_series <- read.csv('level_pig_od_ref.csv')
head(mass_series)

```

```

##    time slurry_mass
## 1     1      67.469
## 2    32      78.591
## 3    60      89.780
## 4    61      87.100
## 5    91      98.289
## 6    92      68.809

```

```
tail(mass_series)
```

```

##    time slurry_mass
## 17  274      39.329
## 18  275      35.309
## 19  304      46.498
## 20  305      45.091
## 21  336      56.280
## 22  365      67.469

```

```

stor_ser_ref <- list(
  type = 'series',
  dat  = mass_series,
  storage_depth = 2,
  area  = 0.7,
  temp_C = 18,
  resid_enrich = 0.2
)

```

```

temp_dat <- data.frame(time = seq(0, 365, length.out = 53))
temp_dat$temp_C <- 10 + 10 * sin((temp_dat$time - 130) / 365 * 2 * pi)

```

```
devtools::load_all()
```

```

## i Loading abmneo
## Loading required package: deSolve

```

```

out_simp_ser <- abmneo(
  storage = stor_ser_ref,
  365,
  inf_pars = inf_VS_dig3,
  grp_pars = grp_VS_dig3,
  sub_pars = sub_VS_dig3,

```

```

var_pars = list(var = temp_dat),
)
head(out_simp_ser)

```

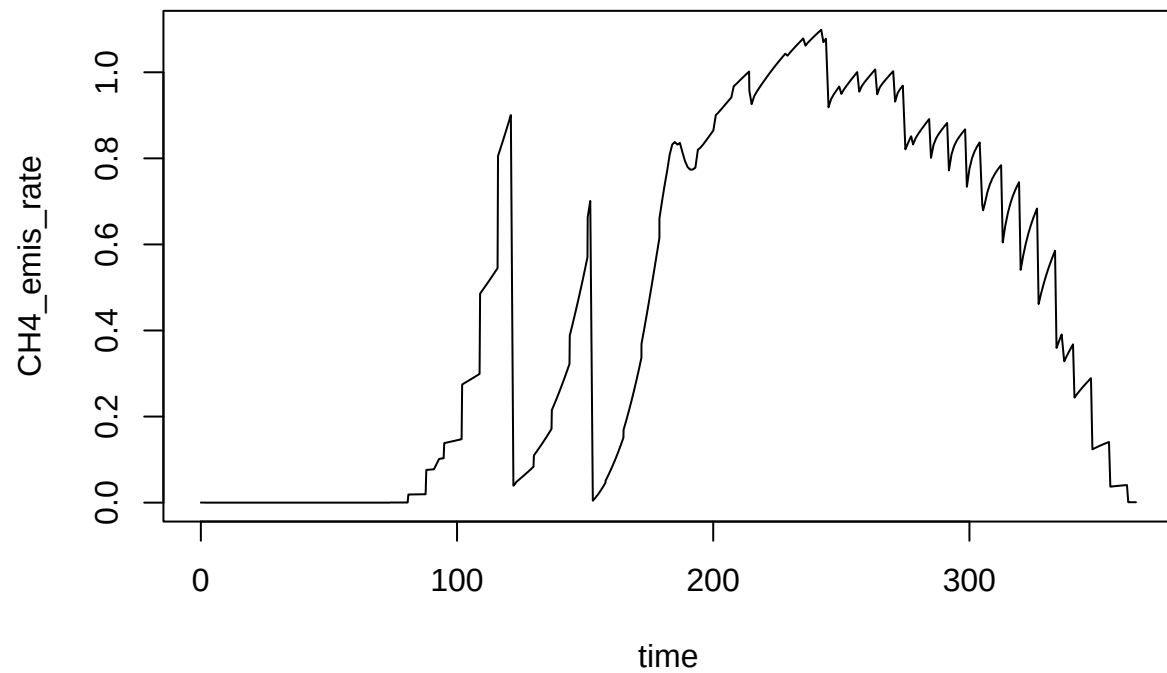
```

##      time      m1      m2      m3      m4      m5      PS      xd
## 1 0.000000 4.048140 4.048140 4.048140 4.048140 4.048140 1229.285 0.000000000
## 2 1.000000 4.047679 4.047668 4.047652 4.047652 4.047652 1220.311 0.002433557
## 3 2.000000 4.068796 4.068784 4.068768 4.068768 4.068768 1218.036 0.004458541
## 4 3.000000 4.089910 4.089899 4.089882 4.089882 4.089882 1215.777 0.006479568
## 5 3.509615 4.100669 4.100658 4.100642 4.100642 4.100642 1214.632 0.007508002
## 6 4.000000 4.111106 4.111095 4.111078 4.111078 4.111078 1213.679 0.008078487
##      VFA slurry_mass      CH4 slurry_load  COD_load CH4_emis_rate
## 1 33.73450    67.46900 0.00000000000    0.0000000    0.000000    0.0001461207
## 2 42.70760    67.46900 0.0001570687    0.0000000    0.000000    0.0001676050
## 3 51.69916    67.82777 0.0001570687    0.3587742    6.823885    0.0000000000
## 4 60.67435    68.18655 0.0001570687    0.7175484   13.647770    0.0000000000
## 5 65.24199    68.36939 0.0001570687    0.9003852   17.125327    0.0000000000
## 6 69.48917    68.54532 0.0001570687    1.0763226   20.471655    0.0000000000
##      temp_C pH m1_eff m2_eff m3_eff m4_eff m5_eff PS_eff xd_eff VFA_eff
## 1 2.143501  8      0      0      0      0      0      0      0      0
## 2 2.143501  8      0      0      0      0      0      0      0      0
## 3 1.947342  8      0      0      0      0      0      0      0      0
## 4 1.947342  8      0      0      0      0      0      0      0      0
## 5 1.947342  8      0      0      0      0      0      0      0      0
## 6 1.455058  8      0      0      0      0      0      0      0      0
##      slurry_mass_eff slurry_depth      m1_conc      m2_conc      m3_conc      m4_conc
## 1      0      0.09638429 0.06000000 0.06000000 0.06000000 0.06000000
## 2      0      0.09638429 0.05999317 0.05999300 0.05999276 0.05999276
## 3      0      0.09689682 0.05998716 0.05998699 0.05998675 0.05998675
## 4      0      0.09740935 0.05998119 0.05998102 0.05998078 0.05998078
## 5      0      0.09767055 0.05997815 0.05997798 0.05997775 0.05997775
## 6      0      0.09792189 0.05997647 0.05997630 0.05997606 0.05997606
##      m5_conc PS_conc      xd_conc VFA_conc m1_eff_conc m2_eff_conc
## 1 0.06000000 18.22000 0.000000e+00 0.5000000      NaN      NaN
## 2 0.05999276 18.08699 3.606926e-05 0.6329959      NaN      NaN
## 3 0.05998675 17.95778 6.573326e-05 0.7622123      NaN      NaN
## 4 0.05998078 17.83016 9.502708e-05 0.8898288      NaN      NaN
## 5 0.05997775 17.76573 1.098153e-04 0.9542573      NaN      NaN
## 6 0.05997606 17.70622 1.178561e-04 1.0137697      NaN      NaN
##      m3_eff_conc m4_eff_conc m5_eff_conc PS_eff_conc xd_eff_conc VFA_eff_conc
## 1      NaN      NaN      NaN      NaN      NaN      NaN
## 2      NaN      NaN      NaN      NaN      NaN      NaN
## 3      NaN      NaN      NaN      NaN      NaN      NaN
## 4      NaN      NaN      NaN      NaN      NaN      NaN
## 5      NaN      NaN      NaN      NaN      NaN      NaN
## 6      NaN      NaN      NaN      NaN      NaN      NaN
##      CH4_emis_rate_ave
## 1      NA
## 2      0.0001570687
## 3      0.0000000000
## 4      0.0000000000
## 5      0.0000000000
## 6      0.0000000000

```

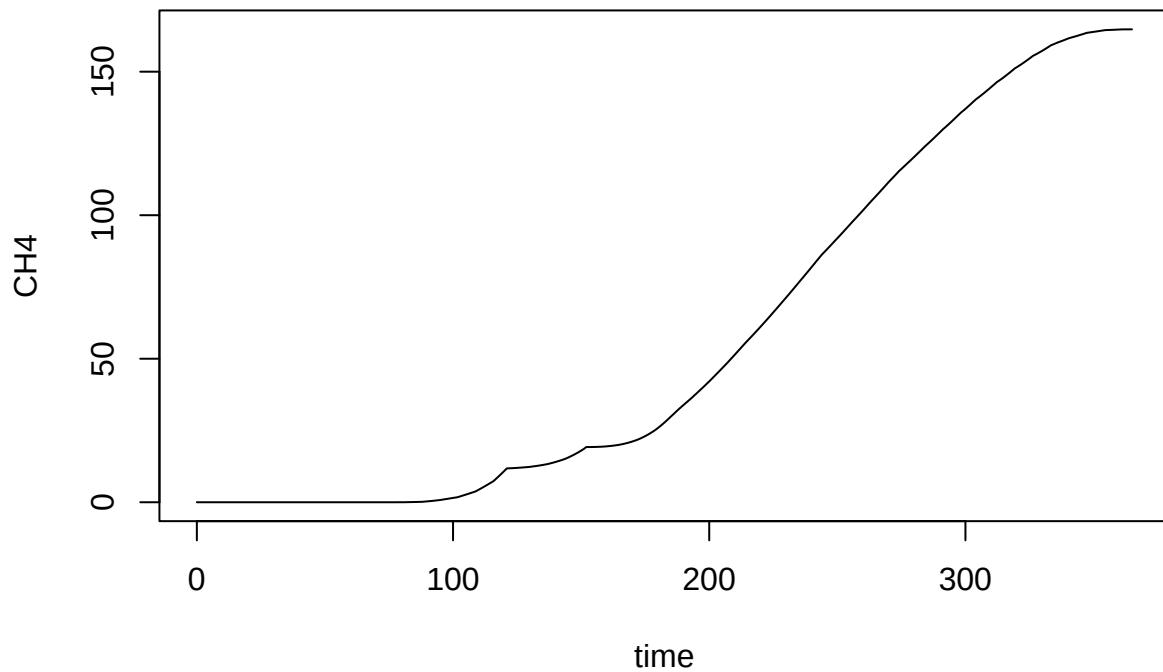
Plots next. Methane emission rate, in g/d.

```
plot(CH4_emis_rate ~ time, data = out_simp_ser, type = 'l')
```



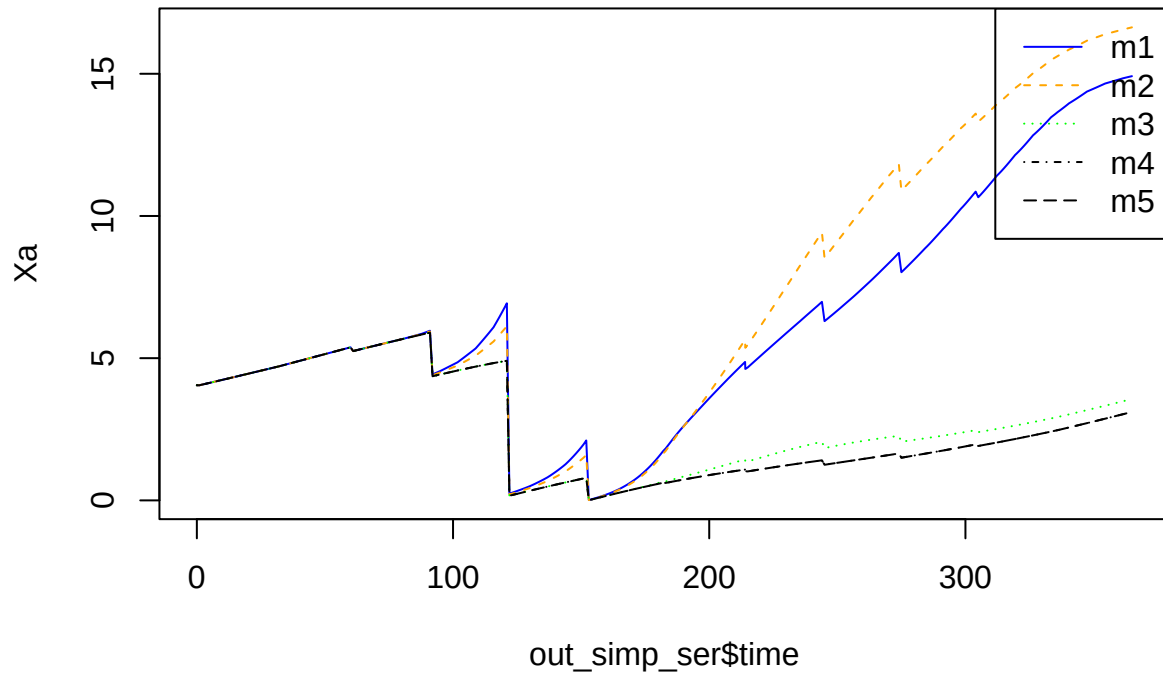
And cumulative emission in g.

```
plot(CH4 ~ time, data = out_simp_ser, type = 'l')
```



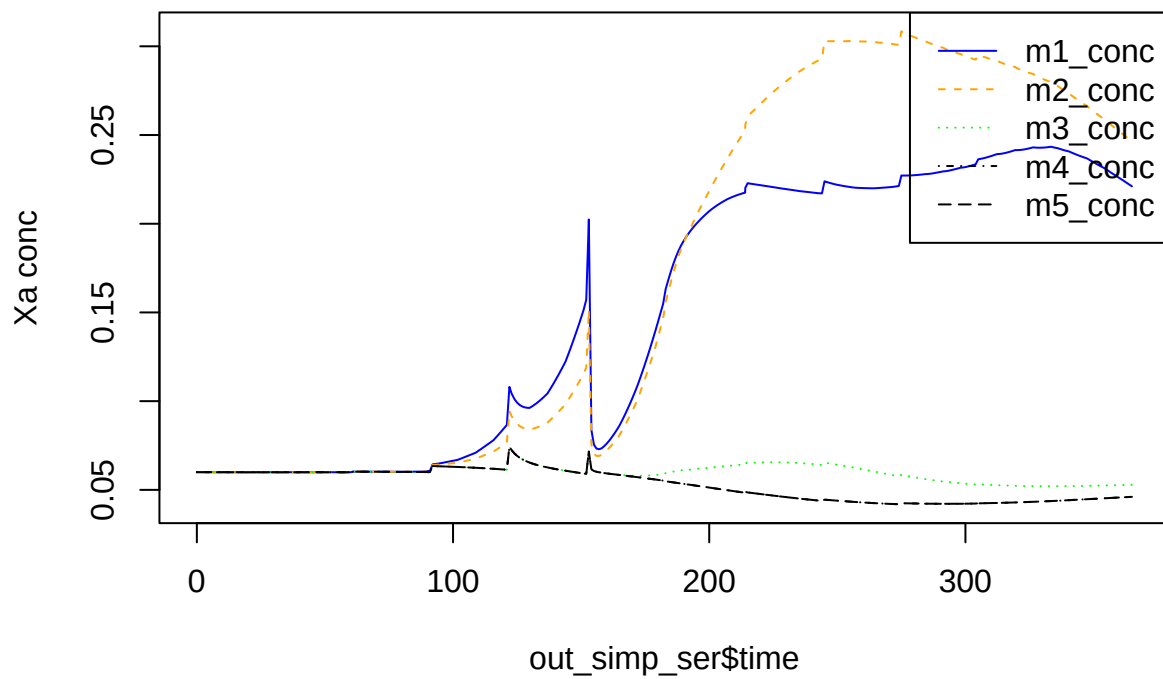
Microbial biomass (g).

```
matplot(
  out_simp_ser$time,
  out_simp_ser[, nn <- grp_VS_dig3$grps],
  col = cc <- c('blue', 'orange', 'green', 'black', 'black', 'black', 'red'),
  lty = 1:length(nn), type = 'l',
  ylab = 'Xa'
)
legend('topright', legend = nn, col = cc, lty = 1:length(nn))
```



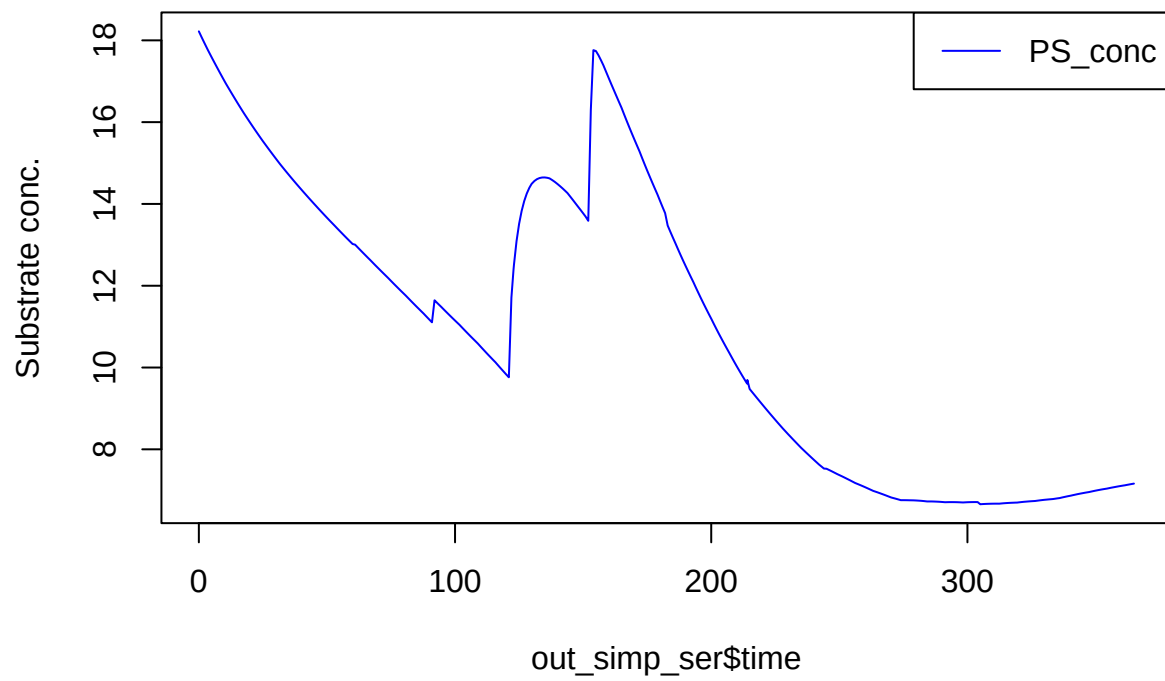
And microbial biomass concentrations (g/kg).

```
matplot(
  out_simp_ser$time,
  out_simp_ser[, nn <- paste0(grp_VS_dig3$grps, '_conc')],
  col = cc <- c('blue', 'orange', 'green', 'black', 'black', 'black', 'red'),
  lty = 1:length(nn),
  type = 'l',
  ylab = 'Xa conc'
)
legend('topright', legend = nn, col = cc, lty = 1:length(nn))
```



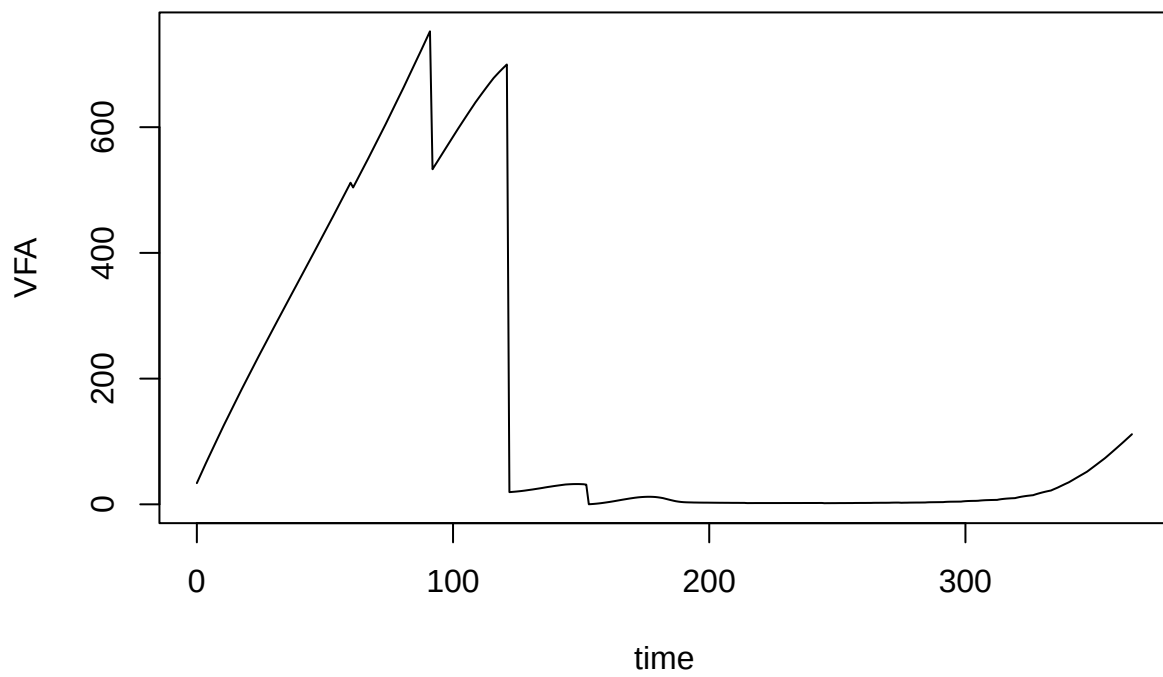
Substrate concentrations.

```
matplot(
  out_simp_ser$time,
  out_simp_ser[, nn <- paste0(sub_VS_dig3$subs, '_conc')],
  col = cc <- c('blue', 'orange', 'green', 'black', 'red'),
  lty = 1:length(nn),
  type = 'l',
  ylab = 'Substrate conc.'
)
legend('topright', legend = nn, col = cc, lty = 1:length(nn))
```

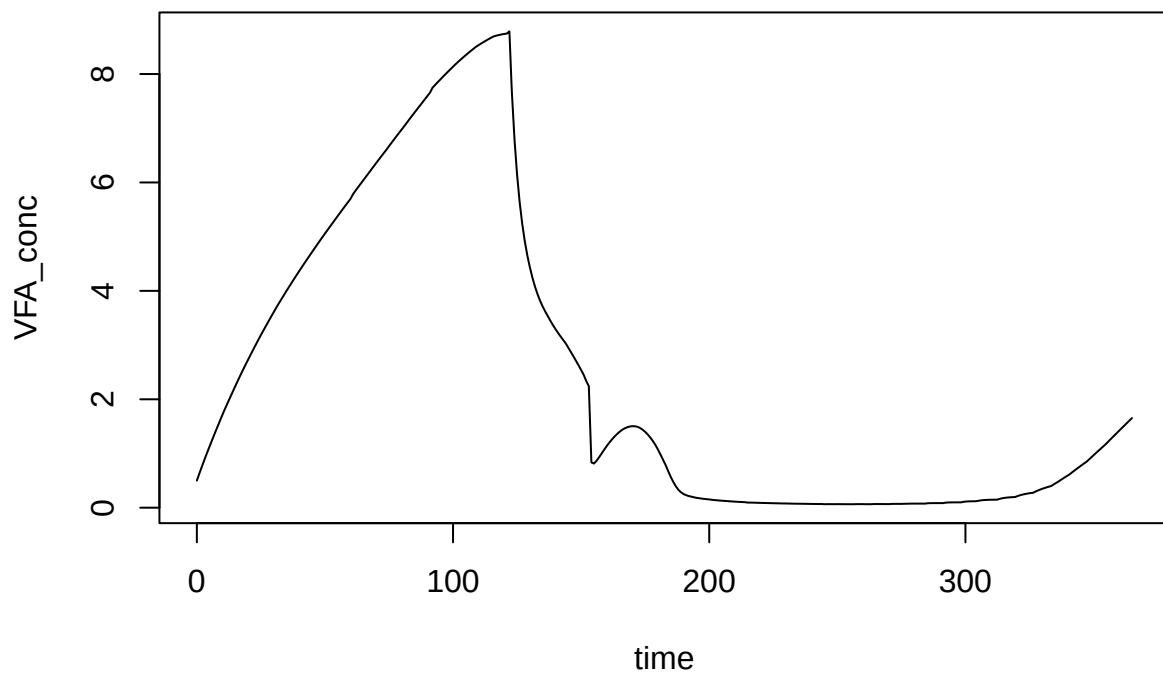


And VFA mass and concentration.

```
plot(VFA ~ time, data = out_simp_ser, type = 'l')
```

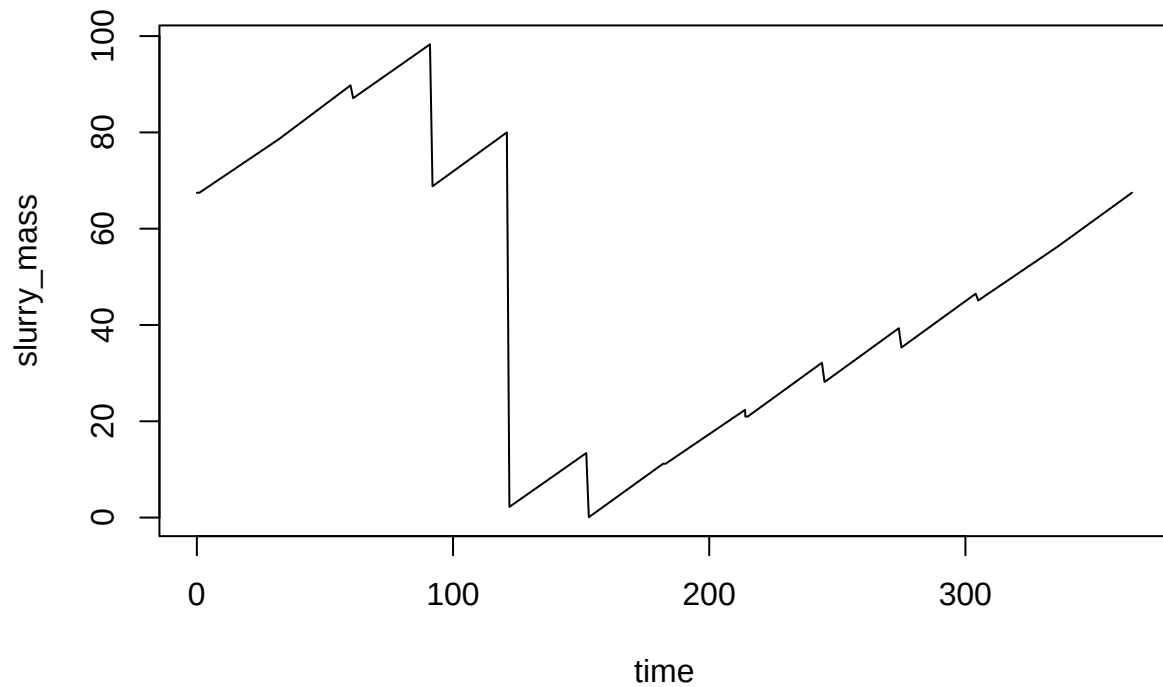



```
plot(VFA_conc ~ time, data = out_simp_ser, type = 'l')
```



Of course most of the temporal patterns are driven by slurry mass, which is also returned.

```
plot(slurry_mass ~ time, data = out_simp_ser, type = 'l')
```



```
names(out_simp_ser)
```

```
## [1] "time"           "m1"             "m2"
## [4] "m3"             "m4"             "m5"
## [7] "PS"             "xd"             "VFA"
## [10] "slurry_mass"    "CH4"            "slurry_load"
## [13] "COD_load"       "CH4_emis_rate"  "temp_C"
## [16] "pH"             "m1_eff"         "m2_eff"
## [19] "m3_eff"         "m4_eff"         "m5_eff"
## [22] "PS_eff"         "xd_eff"         "VFA_eff"
## [25] "slurry_mass_eff" "slurry_depth"   "m1_conc"
## [28] "m2_conc"        "m3_conc"        "m4_conc"
## [31] "m5_conc"        "PS_conc"        "xd_conc"
## [34] "VFA_conc"       "m1_eff_conc"    "m2_eff_conc"
## [37] "m3_eff_conc"    "m4_eff_conc"    "m5_eff_conc"
## [40] "PS_eff_conc"    "xd_eff_conc"    "VFA_eff_conc"
## [43] "CH4_emis_rate_ave"
```